Sequential Circuits and Latches

1. Introduction to Sequential Circuits

Combinational circuits only depend on the current input values to produce outputs. Sequential circuits, however, are digital circuits that store and use previous state information to determine their next state.

- Outputs in a sequential circuit are a function of both the current inputs and the present state of the storage elements.
- The general purpose of these circuits is to store information for an indefinite amount of time.

Applications:

Sequential circuits are commonly used in digital systems to implement:

- State machines
- Timers
- Counters
- Memory elements (implemented using flip-flops, which maintain their binary state even when inputs change)

2. Types of Sequential Circuits

Sequential circuits are divided into two primary categories based on how they handle time and state changes.

Synchronous vs. Asynchronous Circuits

Feature	Synchronous Sequential Circuit	Asynchronous Sequential Circuit
Clock Signal	Uses a clock to synchronize changes.	Does not use a clock; changes occur immediately with input changes.
Operation	All changes happen at specific time intervals (clock pulses).	Changes occur as soon as inputs change, without waiting for a clock.
Speed	Slower due to clock dependency.	Faster because it reacts immediately to input changes.

Feature	Synchronous Sequential Circuit	Asynchronous Sequential Circuit
Design Complexity	Easier to design and analyze because changes happen in sync.	More complex due to potential timing issues (hazards, glitches).
Reliability	More reliable and predictable.	Less reliable due to possible race conditions.

3. Introduction to Latches and Flip-Flops

Synchronous sequential circuits that use clock pulses as inputs for their storage elements are called **clocked sequential circuits**. The simplest form of storage element used in these circuits is the flip-flop.

- **Flip-Flop:** A binary storage device capable of storing one bit of information and having specific timing characteristics.
- **Latches:** The most basic storage elements from which flip-flops are usually constructed.
- Latches store a single bit of information and hold its value until it is updated by new input signals.
- They act as temporary storage elements and can be implemented using various digital logic gates, such as AND, OR, NOT, NAND, and NOR gates.

4. The S-R (Set-Reset) Latch

Set-Reset latches are the simplest form of latches and form the basic building blocks for all other types of flip-flops.

- It is implemented using two inputs: S (Set) and R (Reset).
- The S input sets the output to logic 1, while the R input resets the output to logic 0.
- They are also known as "preset" and "clear" states.
- A basic SR Latch consists of 2 cross-coupled NOR gates or 2 cross-coupled NAND gates, producing 2 outputs: Q and Q'.

Building SR Latch (Using NAND Gate):

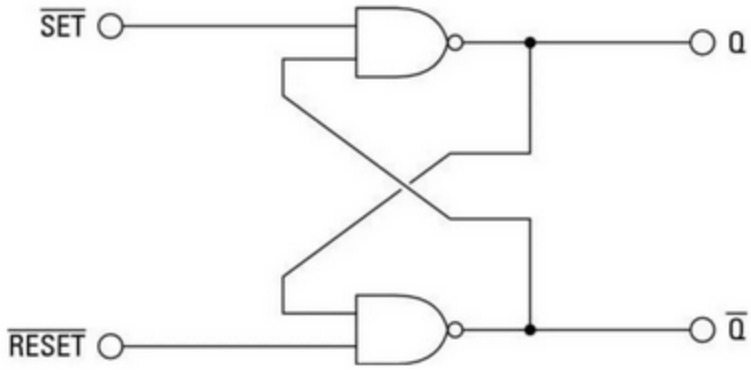
When built with NAND gates, the SR Latch becomes an Active-Low device (often denoted as $\overline{S}$-$\overline{R}$ Latch). This means that a logic 0 triggers the action, and logic 1 is the neutral resting state.

Logic Rules:

- **Hold State ($S' = 1, R' = 1$):** Both inputs are inactive (resting at High). The latch holds its previous value.

- **Set State ($S' = 0, R' = 1$):** Driving S' low forces Q to 1, which feeds back to make $Q' = 0$.
- **Reset State ($S' = 1, R' = 0$):** Driving R' low forces Q' to 1, which feeds back to make $Q = 0$.
- **Invalid State ($S' = 0, R' = 0$):** Both inputs dropping to 0 simultaneously forces both NAND gates to output a 1. Therefore, $Q = 1$ and $Q' = 1$, which violates the inverse rule and creates an undefined state.
  Truth Table:

S'	R'	Q (Next State)
0	0	Invalid (ndefined)
0	1	1 (Set)
1	0	0 (Reset)
1	1	Hold (No Change)



Building SR Latch (Using NOR Gate):
Truth Table When built with NOR gates, the SR Latch is an Active-High device. This means that a logic 1 triggers the Set or Reset action, while logic 0 is the neutral/resting state.

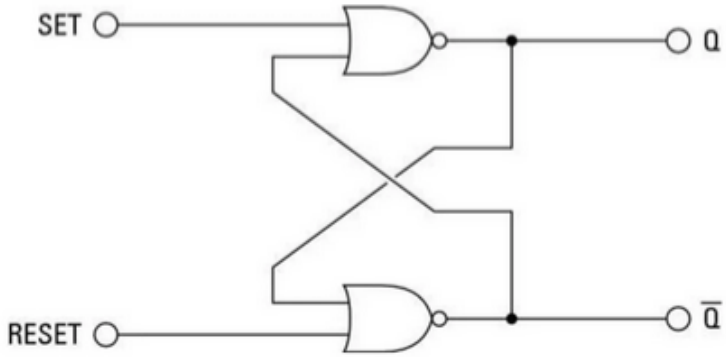
Logic Rules:

- **Hold State ($S = 0, R = 0$):** Both inputs are inactive. The cross-coupled feedback loop maintains whatever value was previously stored in Q.
- **Set State ($S = 1, R = 0$):** The S input forces Q' to 0, which feeds back to make $Q = 1$.
- **Reset State ($S = 0, R = 1$):** The R input forces Q to 0, which feeds back to make $Q' = 1$.
- **Invalid State ($S = 1, R = 1$):** Both inputs attempt to force a zero. This forces both Q and Q' to equal 0 simultaneously. This violates the rule that Q and Q' must be inverses, making it an undefined/invalid state.(Using NOR Gate):
  When built with NOR gates, the SR Latch is an Active-High** device. This means that a logic 1 triggers the Set or Reset action, while logic 0 is the neutral/resting state.

Logic Rules:

- **Hold State ($S = 0, R = 0$):** Both inputs are inactive. The cross-coupled feedback loop maintains whatever value was previously stored in Q.
- **Set State ($S = 1, R = 0$):** The S input forces Q' to 0, which feeds back to make $Q = 1$.
- **Reset State ($S = 0, R = 1$):** The R input forces Q to 0, which feeds back to make $Q' = 1$.
- **Invalid State ($S = 1, R = 1$):** Both inputs attempt to force a zero. This forces both Q and Q' to equal 0 simultaneously. This violates the rule that Q and Q' must be inverses, making it an undefined/invalid state.
  Truth Table:

S'	R'	Q (Next State)
0	0	Hold (No Change)
0	1	0 (Reset)
1	0	1 (Set)
1	1	Invalid (ndefined)



5. S-R Latch with Control Input

Latches can be modified by adding a control input (C), which acts as an enable signal or clock. These controlled latches are used to construct more advanced flip-flops.

Logic Rules:

- When $C = 0$: The control input disables the circuit. The state of the output does not change, regardless of the values of S and R.
- When $C = 1$: The latch operates normally.
- When $C = 1$ and $S = R = 0$: The state holds (no change).
- When $C = 1$ and $S = R = 1$: An undefined state occurs.

Truth Table (SR Latch with Control):

C	S	R	Next state of Q
0	X	X	No change
1	0	0	No change
1	0	1	$Q = 0$ Reset state
1	1	0	$Q = 1$ Set state
1	1	1	ndefined

6. The D (Data) Latch

The D Latch (also called a "Transparent Latch") was designed specifically to eliminate the undesirable undefined state present in the SR latch.

It simplifies the architecture down to only two inputs: D (Data) and C (Control). Internally, the D input is wired directly to the S terminal, and passed through an inverter to the R terminal. This guarantees that S and R can never be 1 at the same time.

Logic Rules:

- The D input is only sampled when $C = 1$.
- If $D = 1$ (and $C = 1$), the Q output goes to 1 (Set state).
- If $D = 0$ (and $C = 1$), the Q output goes to 0 (Reset state).

Truth Table (D Latch):

C	D	Next state of Q
0	X	No change (Hold)
1	0	$Q = 0$ Reset state
1	1	$Q = 1$ Set state

The D Latch Circuit Diagram

